Módulo 1
Fundamentos de Python

Curso:	Python para Ciência de Dados
Público-alvo:	Iniciantes em programação
Ferramenta:	Google Colab / Jupyter Notebook

Alexsandro Lopes - 2323215
Luiz Eduardo Pacheco - 2310314
Gregory Jereissati - 2320425

O que você vai aprender

Uma jornada completa pelos fundamentos da linguagem Python, preparando a base para a Ciência de Dados.

01 Sintaxe básica e primeiro código

02 Variáveis e tipos de dados

03 Operadores

04 Estruturas condicionais

05 Laços de repetição

06 Estruturas de dados

07 Funções

08 Boas práticas de programação

09 Mini-projeto: análise de desempenho de alunos

1. Sintaxe Básica

Python possui uma sintaxe limpa e próxima do inglês. Quatro regras fundamentais:

- **Indentação obrigatória:** blocos são delimitados por espaços, não por chaves {}.
- **Comentários:** qualquer texto após # é ignorado pelo interpretador.
- **Sem ponto-e-vírgula:** cada instrução ocupa uma linha.
- **Case-sensitive:** Nome e nome são variáveis diferentes.

```python
# A função print() exibe mensagens na tela
print("Bem-vindo ao curso de Python!")

# Separador personalizado
print("Olá", "Mundo", sep="-")

# Não quebra a linha no final
print("Linha 1", end=" | ")
print("Linha 2")
```

2. Variáveis e Tipos

Em Python, o tipo é inferido automaticamente — não é necessário declará-lo.

Tipo	Exemplo	Descrição
`int`	22	Número inteiro
`float`	1.75	Número decimal
`str`	"Carlos"	Texto (string)
`bool`	True / False	Valor lógico

```python
nome      = "Carlos"   # str
idade     = 22         # int
altura    = 1.75       # float
estudante = True       # bool

print(f"Aluno: {nome}, {idade} anos, altura {altura}m")
```

Conversão de Tipos (Casting)

```python
float("8.5") → 8.5
int(3.9) → 3
bool(0) → False
```

3. Operadores

Aritméticos

Realizam cálculos matemáticos básicos e avançados:

`+` `-` `*` `/` `//` `%` `**`

Comparação

Comparam valores e sempre retornam `True` ou `False`:

`==` `!=` `>` `<` `>=` `<=`

Lógicos

Combinam múltiplas condições lógicas:

`and` `or` `not`

```python
# Comparação
nota = 7.5
print(nota >= 7)   # True
print(nota == 10)  # False
print(nota != 10)  # True

# Lógicos
freq = 80

# (True) and (True) -> True
aprovado = (nota >= 7) and (freq >= 75)

# (False) or (False) -> False
recupera = (5 <= nota < 7) or (freq < 75)

# not True -> False
nao_passou = not aprovado
```

4. Estruturas Condicionais

Com **if**, **elif** e **else** o programa segue caminhos diferentes conforme a situação.

A estrutura condicional avalia expressões lógicas (verdadeiro ou falso) para decidir qual bloco de código executar, permitindo a tomada de decisões dinâmicas.

> **Forma compacta (ternária):**
> Permite atribuir valores baseados em uma condição em uma única linha, tornando o código mais limpo para decisões simples.

MÚLTIPLAS CONDIÇÕES

```
nota = 6.2

if nota >= 9:
    situacao = "Excelente"
elif nota >= 7:
    situacao = "Aprovado"
elif nota >= 5:
    situacao = "Recuperação"
else:
    situacao = "Reprovado"

print(f"Nota {nota} → {situacao}")
```

CONDICIONAL TERNÁRIA

```
resultado = "Aprovado" if nota >= 7 else "Reprovado"
```

5. Laços de Repetição

- **for** → itera sobre uma sequência conhecida (listas, tuplas, strings).
- **while** → repete um bloco de código enquanto uma condição for verdadeira.

> **List Comprehension**
>
> Forma compacta e pythônica de criar listas baseadas em iteráveis existentes, substituindo laços for simples.

```python
# Calculando a média da turma
notas = [8.5, 6.0, 9.0, 5.5, 7.5]
soma = 0

for nota in notas:
soma += nota

media = soma / len(notas)
print(f"Média: {media:.2f}")

# enumerate() → índice + valor
for i, aluno in enumerate(["Ana", "Bruno"], start=1):

print(f"{i}. {aluno}")
```

```python
# List comprehension
aprovadas = [n for n in notas if n >= 7]
```

6. Estruturas de Dados

Estrutura	Símbolo	Ordenada	Mutável	Duplicatas
Lista (`list`)	`[]`	Sim	Sim	Sim
Tupla (`tuple`)	`()`	Sim	Não	Sim
Dicionário (`dict`)	`{}`	Sim*	Sim	Chaves: Não
Conjunto (`set`)	`{}`	Não	Sim	Não

**ordenado por inserção desde Python 3.7*

```python
# Dicionário → equivale a uma linha de um dataset
aluno = {"nome": "Marina", "nota_final": 8.5, "aprovado": True}

print(aluno["nome"])                          # acesso direto
print(aluno.get("email", "não informado")) # acesso seguro

# set → encontrar valores únicos
categorias_unicas = set(["A", "B", "A", "C", "B"])
# → {'A', 'B', 'C'}
```

7. Funções

Funções encapsulam lógica reutilizável e evitam repetição de código — um princípio fundamental de boas práticas.

Parâmetros Padrão

Permitem definir valores default para argumentos, tornando-os opcionais na chamada da função.

Múltiplos Retornos

Em Python, uma função pode retornar vários valores simultaneamente (empacotados como uma tupla).

FUNÇÃO BÁSICA

```
def classificar_nota(nota):

"""Classifica uma nota."""

if nota >= 9:   return "Excelente"

elif nota >= 7: return "Aprovado"

elif nota >= 5: return "Recuperação"
```

PARÂMETROS COM VALOR PADRÃO

```
def calcular_media(notas, arredondar=2):

return round(sum(notas) / len(notas), arredondar)
```

MÚLTIPLOS RETORNOS

```
def resumo_estatistico(notas):

return round(sum(notas)/len(notas), 2), min(notas), max(notas)

# Desempacotando a tupla retornada
media, minimo, maximo = resumo_estatistico([8.5, 6.0, 9.0])
```

8. Boas Práticas — PEP 8

Nomenclatura

- `snake_case` → variáveis e funções:
  nota_do_aluno , calcular_media()
- `UPPER_CASE` → constantes:
  NOTA_MINIMA_APROVACAO = 7.0
- `PascalCase` → classes:
  RelatorioDesempenho

Comentários Úteis

> Explique o **porquê**, não o **o quê** óbvio. O código deve ser legível por si só.

Evite Números Mágicos

```
# ✗ Ruim — o que significa 7 aqui?
aprovados = [n for n in notas if n >= 7]
```

```
# ✓ Bom — a constante documenta a regra
NOTA_MINIMA_APROVACAO = 7.0

aprovados = [n for n in notas if n >=
NOTA_MINIMA_APROVACAO]
```

9. Mini-Projeto

Objetivo

Calcular estatísticas, classificar alunos e exibir um relatório formatado usando apenas Python puro, integrando todos os conceitos aprendidos.

Dataset Simulado:

Lista de dicionários contendo 8 alunos com os seguintes campos:

- nome (str)
- nota_final (float)
- frequencia (int)

```
============================================================
ALUNO        NOTA   FREQ  SITUAÇÃO
============================================================
Ana           8.5    90%  Aprovado
Bruno         6.0    75%  Recuperação
Carla         9.0    95%  Excelente
Diego         5.5    68%  Reprovado por falta
Elaine        7.5    88%  Aprovado
Felipe        4.0    60%  Reprovado por falta
Gia           9.5    98%  Excelente
Hugo          3.5    50%  Reprovado por falta
============================================================

📊 RESUMO DA TURMA
------------------------------
Média da turma : 6.69
Aprovados      : 4 aluno(s)
Em recuperação : 1 aluno(s)
Reprovados     : 3 aluno(s)
------------------------------
Taxa de aprovação: 50.0%
```

Conclusão do Módulo 1

Tópico	O que aprendemos
Sintaxe básica	`print()`, comentários, indentação
Variáveis e tipos	`int`, `float`, `str`, `bool`, f-strings, conversão
Operadores	Aritméticos, comparação, lógicos
Condicionais	`if`, `elif`, `else`, ternária
Laços	`for`, `while`, `break`, `continue`, list comprehension
Estruturas de dados	`list`, `dict`, `tuple`, `set`
Funções	`def`, parâmetros, retorno, docstrings
Boas práticas	PEP 8, nomes descritivos, constantes, comentários úteis

Próximo Passo: Módulo 2

Aprenderemos a usar a biblioteca **Pandas** para carregar, limpar e transformar conjuntos de dados reais de forma muito mais eficiente do que com Python puro.

Pratique: resolva a lista de exercícios do Módulo 1 antes de avançar!
Disponível no NOSSO GITHUB